```
person = Person.new(name: "JD")
person.valid? # => false
person.errors[:name] # => ["is too short (minimum is 3 characters)"]

person = Person.new
person.valid? # => false
person.errors[:name]
 # => ["can't be blank", "is too short (minimum is 3 characters)"]
```

7.3 errors.add

The add method lets you manually add messages that are related to particular attributes. You can use the errors.full_messages or errors.to_a methods to view the messages in the form they might be displayed to a user. Those particular messages get the attribute name prepended (and capitalized). add receives the name of the attribute you want to add the message to, and the message itself.

```
class Person < ActiveRecord::Base
  def a_method_used_for_validation_purposes
    errors.add(:name, "cannot contain the characters !@#%*()_-+=")
  end
end

person = Person.create(name: "!@#")

person.errors[:name]
 # => ["cannot contain the characters !@#%*()_-+="]

person.errors.full_messages
 # => ["Name cannot contain the characters !@#%*()_-+="]
```

Another way to do this is using []= setter

```
class Person < ActiveRecord::Base
  def a_method_used_for_validation_purposes
    errors.add(:name, "cannot contain the characters !@#%*()_-+=")
  end
end

person = Person.create(name: "!@#")

person.errors[:name]
 # => ["cannot contain the characters !@#%*()_-+="]

person.errors.to_a
 # => ["Name cannot contain the characters !@#%*()_-+="]
```

7.4 errors.details

You can specify a validator type to the returned error details hash using the errors.add method.

```
class Person < ActiveRecord::Base
  def a_method_used_for_validation_purposes
    errors.add(:name, :invalid_characters)
  end
end

person = Person.create(name: "!@#")

person.errors.details[:name]
# => [{error: :invalid_characters}]
```

To improve the error details to contain the unallowed characters set for instance, you can pass additional keys to errors.add.

```
class Person < ActiveRecord::Base
```